Symfony Security Review

Educational Bug Bounty Series • Authorized Security Research Only

■ Symfony Security Review

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope: *Review only authorized Symfony applications. Do not retrieve configuration secrets, execute code, or access private profiler data.*

■ Executive Summary

Misconfigured Symfony deployments may expose development tools, profiler endpoints, configuration files, or sensitive debug information. The supplied one-page checklist references Symfony RCE research, the `eos` tool, sensitive-path examples, secret-fragment research, template-checklist files, and a Symfony wordlist for fuzzing.

■ Core Testing Methodology

1. Confirm the target is in scope and identify the Symfony version where safely possible.
2. Review deployment configuration to confirm production mode is enabled and debug features are disabled.
3. Check, with minimal requests, whether development profiler or debug routes are publicly accessible.
4. Verify configuration and test files are not served by the web server.
5. Check that error pages do not disclose paths, environment variables, secrets, or stack traces.
6. Review templates, routing, and user-controlled input handling through code/configuration review or a staging environment.
7. Compare the installed version and dependencies with official security advisories.
8. Avoid secret-fragment or RCE exploitation on live targets; use isolated labs for validation.
9. Record minimal redacted evidence and retest.

■ High-Value Bug Bounty Tips & Edge Cases

- The source lists profiler/debug-style paths and configuration/test-file examples as sensitive exposure categories. These are checks, not confirmation of exposure.
- Development tools must not be available in production.
- Template injection and RCE are high-impact classes; assess safely via code review or isolated testing.
- The source references `eos`, secret-fragment research, template checklists, and a Symfony fuzzing wordlist; it does not provide a complete verified exploit procedure.
- Fuzzing can create load and noisy results; obtain explicit authorization and use a conservative rate.

■■ Educational Checklists & Safe Examples

```
Application: [Authorized-Symfony-Host]
Environment: [Production / Staging]
Version: [Verified-Version]
Profiler/debug routes exposed: [Yes/No/Inconclusive]
Config/test files publicly served: [Yes/No/Inconclusive]
Sensitive errors: [Yes/No]
Evidence: [Redacted]
```

- [ ] Production debug mode is disabled.
- [ ] Profiler and development routes are restricted.
- [ ] Config, secrets, and test files are outside public web root.
- [ ] Templates treat user input as data.

- [ ] Dependencies are patched.
- [ ] Error pages do not expose internals.

■■ Mitigation & Remediation

Set production environment and disable debug mode, restrict profiler routes, keep configuration and secrets outside public directories, use least-privilege file permissions, patch Symfony and dependencies, avoid unsafe template evaluation, and add deployment checks to prevent development artifacts from shipping.

■ Source Notes

Based on the supplied one-page *Symfony.pdf*. It lists RCE research, an `eos` tool reference, profiler/configuration path categories, secret-fragment research, template-checklist references, and a Symfony wordlist. No exploit result or affected version is established by the source.

Educational use only. Test within authorized scope.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.